

KiwiTrack

Attendance Management System

Complete Technical Documentation

React 18

Node.js

Express

SQLite

Vite

Tailwind CSS

Version:	1.0.0	Stack:	MERN-like (React + Node + SQLite)
Backend:	Node.js + Express	Frontend:	React 18 + Vite + Tailwind CSS
Database:	SQLite 3 (WAL)	Auth:	JWT Bearer Token (8h expiry)
Port:	Backend: 5000	Port:	Frontend: 5173

Table of Contents

1.	Project Overview	Architecture & goals
2.	Technology Stack	All libraries & versions
3.	Project Structure	Folder & file layout
4.	Database Schema	All 13 tables documented
5.	Backend Architecture	Express server, routes, middleware
6.	API Reference	All REST endpoints
7.	Authentication Flow	JWT, login, OTP reset
8.	Frontend Architecture	React, routing, state, Axios
9.	Core Features	Attendance, Leave, Payslip, etc.
10.	Verification Checkout System	Random popup & Service Worker
11.	Company Settings	Configurable settings catalogue
12.	Data Flow & Request Pipeline	End-to-end request lifecycle
13.	Environment & Deployment	Setup, .env, scripts

1. Project Overview

Architecture, Goals & System Design

What is KiwiTrack?

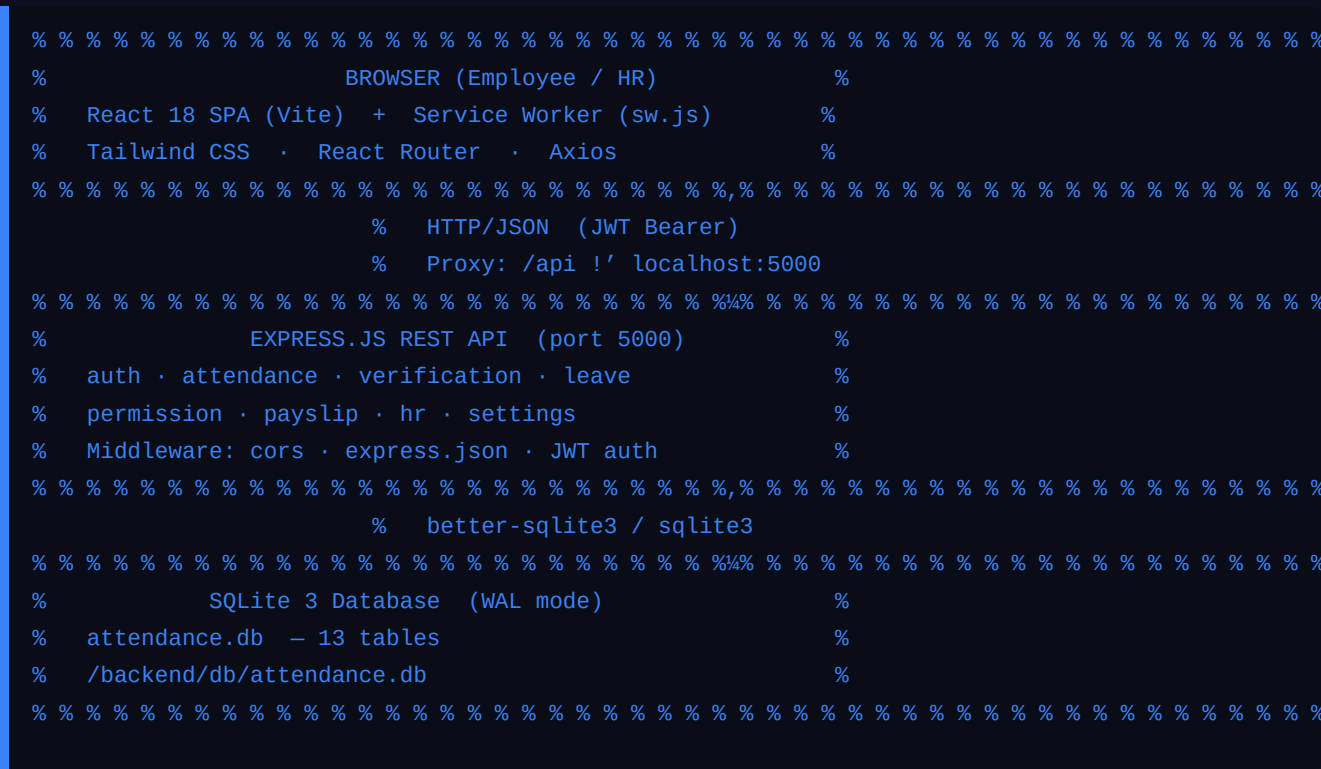
KiwiTrack is a full-stack web-based Employee Attendance Management System designed for small-to-medium businesses. It provides real-time attendance tracking, leave management, payslip distribution, and a unique random verification checkout mechanism that ensures employees are genuinely present at their workstations.

The system has two user roles: Employee (logs attendance, applies for leave/permission, views payslips) and HR (manages employees, approves requests, configures company settings, views analytics).

Core Design Principles

- Single-page application (SPA) — React on Vite, no full-page reloads
- RESTful JSON API — stateless Express backend, JWT-authenticated
- File-based SQLite database — zero-config, WAL mode for concurrent reads
- Role-based access control (RBAC) — employee vs HR middleware guards
- Progressive notification system — in-app popup + OS notification via Service Worker
- Configurable rules — all thresholds (late time, session windows, grace periods) stored in DB

High-Level Architecture



User Roles

Role	Access Level	Key Permissions
employee	Authenticated	Check in/out, view own attendance, apply leave & permission, view payslips
hr	Authenticated + requireHR	All employee data, approve/reject requests, manage employees, upload navslins, configure settings
admin	Same as HR	Full access — identical to HR role in current implementation

2. Technology Stack

Every library, version, and purpose

Frontend Dependencies

Package	Version	Purpose
react	18.3.1	UI component library — hooks-based, functional components throughout
react-dom	18.3.1	DOM renderer for React
react-router-dom	6.26.1	Client-side routing — nested routes, protected routes, outlet pattern
axios	1.7.3	HTTP client — interceptors for JWT attachment & 401 redirect
react-hot-toast	2.4.1	Toast notification library for success/error feedback
vite	5.2.0	Build tool & dev server — HMR, fast cold starts
@vitejs/plugin-react	4.3.1	Vite plugin — Babel-based React JSX transform
tailwindcss	3.4.10	Utility-first CSS framework — all styles via class names
postcss	8.4.41	CSS post-processor — required by Tailwind
autoprefixer	10.4.20	Adds vendor prefixes automatically

Backend Dependencies

Package	Version	Purpose
express	4.18.3	HTTP server framework — routing, middleware chain
sqlite3	5.1.7	SQLite database driver with async callback API
bcryptjs	2.4.3	Password hashing — bcrypt with salt rounds 10
jsonwebtoken	9.0.2	JWT signing & verification — HS256 algorithm
cors	2.8.5	Cross-origin resource sharing middleware
dotenv	16.4.5	Loads .env into process.env at startup
multer	1.4.5-lts.1	Multipart form-data — PDF payslip file uploads
nodemailer	6.9.13	SMTP email — OTP delivery for password reset
nodemon	3.1.0 (dev)	Auto-restart server on file save during development
pdfkit	latest (dev)	PDF generation for project documentation

Browser / Runtime APIs Used

API	Used For	Notes
Web Notifications API	OS-level alerts for verification checkout	Requires user permission grant
Service Worker API	Background notification click handling	sw.js in /public — enables tab focus from any app
Cache API	SW auth token storage	JWT stored so SW can call /api/verification/respond
Web Audio API	Legacy — replaced by Audio element	Now uses marimba-notification WAV file
Audio element	Notification sound playback	/public/notification.wav — preloaded on mount
localStorage	JWT token + user object persistence	Cleared on 401 or logout
document.hidden	Tab visibility detection for OS notifications	Combined with document.hasFocus()
visibilitychange event	Immediate poll on tab switch	Fires when user tabs back in

3. Project Structure

Folder layout and file responsibilities

Root Layout

```
attendance-system/
% % % backend/                                !• Express.js API server
%   % % % server.js                          !• Entry point – mounts all routes
%   % % % .env                               !• Secrets (JWT, SMTP, port)
%   % % % .env.example                       !• Template for new setups
%   % % % package.json
%   % % % db/
%   %   % % % database.js                    !• SQLite connection + promise helpers
%   %   % % % migrate.js                     !• v1 schema (all base tables)
%   %   % % % migrate_v2.js                  !• attendance_logs, audit_logs
%   %   % % % migrate_v3.js                  !• verification_checks table
%   %   % % % migrate_v4.js                  !• login_history, settings expansion
%   %   % % % migrate_v5.js                  !• session window settings
%   %   % % % seed.js                        !• Dev seed data
%   %   % % % indianHolidays.js             !• Static Indian govt holiday list
%   %   % % % attendance.db                  !• SQLite database file (WAL mode)
%   % % % middleware/
%   %   % % % auth.js                        !• JWT auth + requireHR + requireEmployee
%   % % % routes/
%   %   % % % auth.js                        !• Login, me, change-pw, OTP reset
%   %   % % % attendance.js                  !• Check-in, check-out, today, my, summary
%   %   % % % verification.js                !• Poll, respond, schedule, HR views
%   %   % % % verificationHelper.js          !• Shared scheduling logic
%   %   % % % leave.js                       !• Types, balances, apply, approve/reject
%   %   % % % permission.js                  !• Apply, my, all, approve/reject
%   %   % % % payslip.js                     !• Upload, employee view, download
%   %   % % % hr.js                          !• Employees CRUD, attendance grid, dashboard
%   %   % % % settings.js                    !• company_settings GET/PUT
%   % % % uploads/
%       % % % payslips/{userId}/             !• Uploaded PDF payslips
```

Frontend Layout

```
frontend/
% % % index.html          !• Single HTML shell
% % % vite.config.js      !• Vite config + /api proxy to :5000
% % % tailwind.config.js
% % % postcss.config.js
% % % package.json
% % % public/
%   % % % favicon.svg
%   % % % sw.js           !• Service Worker (notification click handler)
%   % % % notification.wav !• Marimba alert sound
% % % src/
%   % % % main.jsx        !• React entry – BrowserRouter + ThemeProvider
%   % % % App.jsx         !• Route tree + VerificationModal (global)
%   % % % index.css       !• CSS variables, base styles
%   % % % api/
%     % % % axios.js      !• Axios instance + interceptors
%   % % % context/
%     % % % AuthContext.jsx !• user, login(), logout() – React Context
%     % % % ThemeContext.jsx !• dark/light mode toggle
%   % % % components/
%     % % % EmployeeLayout.jsx !• Sidebar + outlet for employee portal
%     % % % HRLayout.jsx      !• Sidebar + outlet for HR portal
%     % % % ProtectedRoute.jsx !• Role-based route guard
%     % % % VerificationModal.jsx !• Global checkout popup + SW integration
%     % % % Modal.jsx         !• Reusable modal wrapper
%     % % % LoadingSpinner.jsx
%     % % % ThemeToggle.jsx
%   % % % pages/
%     % % % Login.jsx
%     % % % employee/
%       % % % Dashboard.jsx !• Today status, calendar, summary cards
%       % % % Attendance.jsx !• Monthly view with log detail
%       % % % Leave.jsx      !• Apply & history
%       % % % Permission.jsx !• Apply & history
%       % % % Payslips.jsx   !• Download payslips
%       % % % Settings.jsx   !• Password change, notification prefs
%     % % % hr/
%       % % % Dashboard.jsx !• KPIs, today stats
%       % % % Employees.jsx !• Add/edit/delete employees
%       % % % Attendance.jsx !• Monthly grid, all employees
%       % % % Leave.jsx      !• Approve/reject leave requests
%       % % % Permission.jsx !• Approve/reject permission requests
%       % % % Payslips.jsx   !• Upload payslips per employee
%       % % % Holidays.jsx   !• Manage + auto-sync Indian holidays
%       % % % AuditLogs.jsx  !• HR action log
%       % % % Settings.jsx   !• All company_settings configuration
```

4. Database Schema

SQLite — 13 tables, WAL mode enabled

Database Overview

KiwiTrack uses SQLite 3 with WAL (Write-Ahead Logging) mode enabled for better concurrent read performance. The database file is stored at backend/db/attendance.db and is migrated in 5 versioned scripts (migrate.js through migrate_v5.js).

All primary keys are AUTOINCREMENT integers. Foreign keys are enforced via PRAGMA foreign_keys = ON. Timestamps are stored as ISO 8601 strings (datetime("now") default).

Table	Purpose	Key Columns
users	All accounts (employees + HR)	id, name, email, password_hash, role, employee_id, department, status
attendance	Daily attendance summary row	user_id, date, status, first_check_in, last_check_out, total_working_minutes
attendance_logs	Individual check-in/out events	attendance_id, user_id, date, action, timestamp, source
leave_types	CL, SL, EL etc. definitions	id, name, max_per_year, is_paid
leave_balances	Per-employee per-year balances	user_id, leave_type_id, year, allocated, used, remaining
leave_requests	Leave applications	user_id, leave_type_id, start_date, end_date, days, status, approved_by
permission_requests	Short absence requests	user_id, date, from_time, to_time, hours, status
salary_structures	Salary components per employee	user_id, basic, hra, allowances, deductions, pf, net_pay
payslips	Uploaded PDF metadata	user_id, month, year, file_path, net_pay
holidays	Company calendar holidays	date, name, type (national/gazetted/restricted), category
company_settings	Key-value config store	key (PK), value
verification_checks	Random checkout trigger records	user_id, date, session, scheduled_time, status, response_delay_seconds
audit_logs	HR action history	actor_id, action, target_table, target_id, details
login_history	Login IP + user-agent log	user_id, ip_address, user_agent, created_at

Attendance Status Values

Status	Meaning	Set When
present	On time full day	First check-in before <code>late_threshold_time</code>
late	Checked in late	First check-in after <code>late_threshold_time</code> but before <code>half_day_threshold_time</code>
half_day	Half day worked	First check-in after <code>half_day_threshold_time</code> OR total minutes < <code>min_full_day_minutes_at_checkout</code>
absent	No attendance	Default — no check-in recorded for the day
on_leave	Leave approved	Automatically set when HR approves a leave request covering this date
holiday	Public holiday	Set when holiday is added to the holidays table for this date
weekend	Weekend day	Saturday or Sunday (configurable via <code>weekend_days</code> setting)

Verification Check Status Values

Status	Meaning
pending	Trigger exists but <code>scheduled_time</code> has not been reached yet, OR reached but employee has not responded
responded	Employee clicked Check In within the grace period — <code>checkin_time</code> and <code>response_delay_seconds</code> recorded
no_response	Grace period elapsed without response — automatically marked by next poll call

company_settings Keys Reference

Key	Default	Description
work_start_time	09:30	Official work start time
late_threshold_time	10:00	Check-ins after this are "late"
half_day_threshold_time	12:30	Check-ins after this are "half day"
work_end_time	18:30	Official work end time
min_full_day_minutes	270	Min working minutes to count as full day
verification_enabled	1	Master on/off toggle for random checkout
verification_checks_per_session	2	How many triggers per session window
verification_min_gap_minutes	30	Minimum gap between two triggers
verification_grace_period_minutes	5	Minutes to respond before "no response"
verification_exclude_start_minutes	15	Buffer at start of session window
verification_exclude_end_minutes	15	Buffer at end of session window
verification_session1_start	10:00	Session 1 window start
verification_session1_end	13:00	Session 1 window end
verification_session2_start	14:30	Session 2 window start
verification_session2_end	18:30	Session 2 window end
weekend_days	0,6	0=Sunday, 6=Saturday
max_permissions_per_month	4	Max permission requests per month

5. Backend Architecture

Express.js server design and middleware chain

Server Startup & Middleware Chain

```
// server.js – request flows through this chain:

app.use(cors({ origin: "http://localhost:5173", credentials: true })))
app.use(express.json()) // parse JSON bodies
app.use(express.urlencoded({...})) // parse form bodies
app.use("/uploads", express.static(...)) // payslip file serving

// Route mounting
app.use("/api/auth", authRouter)
app.use("/api/attendance", attendanceRouter)
app.use("/api/verification", verificationRouter)
app.use("/api/leave", leaveRouter)
app.use("/api/permission", permissionRouter)
app.use("/api/payslip", payslipRouter)
app.use("/api/hr", hrRouter)
app.use("/api/settings", settingsRouter)
```

Authentication Middleware

Every protected route passes through auth.js before the route handler runs:

```
// middleware/auth.js

const auth = async (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1]
  const decoded = jwt.verify(token, process.env.JWT_SECRET)
  const user = await db.getAsync("SELECT * FROM users WHERE id=? AND status=?",
    [decoded.id, "active"])
  req.user = user // attached to every subsequent handler
  next()
}

const requireHR = (req, res, next) => {
  if (req.user.role !== "hr" && req.user.role !== "admin")
    return res.status(403).json({ message: "HR access required" })
  next()
}
```

Database Helpers (Promisified SQLite)

SQLite3 is callback-based. database.js wraps the three most-used methods with Promises:

```
db.getAsync(sql, params)    // !' Promise<row | undefined>
db.allAsync(sql, params)    // !' Promise<row[]>
db.runAsync(sql, params)    // !' Promise<{ id, changes }>

// WAL mode for concurrent reads:
db.run("PRAGMA journal_mode = WAL")
db.run("PRAGMA foreign_keys = ON")
```

6. API Reference

All REST endpoints with methods and auth requirements

Authentication /api/auth

Method	Endpoint	Auth	Description
POST	/api/auth/login	None	Email + password !' JWT token + user object
GET	/api/auth/me	JWT	Returns current user (without password_hash)
POST	/api/auth/change-password	JWT	Changes password — requires current password
POST	/api/auth/forgot-password	None	Sends 6-digit OTP to email via SMTP
POST	/api/auth/verify-otp	None	Validates OTP without resetting password
POST	/api/auth/reset-password	None	OTP + new password !' resets password, clears OTP

Attendance /api/attendance

Method	Endpoint	Auth	Description
POST	/api/attendance/check-in	JWT	Records check-in, schedules verification triggers
POST	/api/attendance/check-out	JWT	Records check-out, recomputes working minutes
GET	/api/attendance/today	JWT	Today's attendance row + all log entries
GET	/api/attendance/my?month=&year=	JWT	Employee's monthly attendance records
GET	/api/attendance/summary?month=&year=	JWT	Count by status for the month

Verification /api/verification

Method	Endpoint	Auth	Description
GET	/api/verification/poll	JWT	Polls for pending trigger — triggers auto-checkout, returns data for non-in
POST	/api/verification/respond	JWT	Employee checks in after popup — records delay, inserts check-in log
POST	/api/verification/schedule	JWT	Manually schedule triggers (called internally by check-in)
GET	/api/verification/today	JWT	Today's verification checks for current employee
GET	/api/verification/hr/:userId/:date	JWT+HR	Per-day checks for HR view
GET	/api/verification/hr/:userId/summary	JWT+HR	Monthly stats — avg delay, no-response count
GET	/api/verification/hr/grid?year=&month=	JWT+HR	Flags cells in HR attendance grid

Leave /api/leave

Method	Endpoint	Auth	Description
GET	/api/leave/types	JWT	All leave types (CL, SL, EL...)
GET	/api/leave/balances	JWT	Employee's own leave balances for year
POST	/api/leave/apply	JWT	Submit leave request — checks balance & overlap
GET	/api/leave/my	JWT	Employee's own leave request history
GET	/api/leave/all?status=	JWT+HR	All requests filtered by status
PUT	/api/leave/:id/approve	JWT+HR	Approve — deducts balance, marks attendance on leave
PUT	/api/leave/:id/reject	JWT+HR	Reject — no balance change

HR /api/hr

Method	Endpoint	Auth	Description
GET	/api/hr/employees	JWT+HR	All active employees with salary info
POST	/api/hr/employees	JWT+HR	Create employee + salary + leave balances
PUT	/api/hr/employees/:id	JWT+HR	Update employee details + salary
DELETE	/api/hr/employees/:id	JWT+HR	Permanently delete employee (cascades all data)
PUT	/api/hr/employees/:id/deactivate	JWT+HR	Toggle active/inactive status
GET	/api/hr/attendance?month=&year=	JWT+HR	Full attendance grid for all employees
GET	/api/hr/attendance/:userId/:date	JWT+HR	Single employee single day detail
GET	/api/hr/dashboard	JWT+HR	KPI counts — present, absent, leaves, permissions
GET	/api/hr/audit-logs	JWT+HR	Last 100 HR actions
GET	/api/hr/holidays?year=	JWT	Holiday list for year (all roles)
POST	/api/hr/holidays	JWT+HR	Add single holiday
POST	/api/hr/holidays/sync/:year	JWT+HR	Auto-import Indian govt holidays for year
DELETE	/api/hr/holidays/:id	JWT+HR	Remove holiday

Settings /api/settings

Method	Endpoint	Auth	Description
GET	/api/settings	JWT	Returns all company_settings as key-value object
PUT	/api/settings	JWT+HR	Bulk update — accepts object of key:value pairs

Payslip /api/payslip

Method	Endpoint	Auth	Description
POST	/api/payslip/upload/:userId	JWT+HR	Upload PDF (multipart) — stored in uploads/payslips/{userId}/
GET	/api/payslip/my	JWT	Employee's own payslip list
GET	/api/payslip/:id/download	JWT	Stream PDF file to browser
GET	/api/payslip/hr/:userId	JWT+HR	All payslips for a specific employee

7. Authentication Flow

JWT lifecycle, OTP password reset, route guards

Login Flow

1. User submits email + password to POST /api/auth/login
2. Backend: `bcrypt.compare(password, user.password_hash)`
3. On success: `jwt.sign({ id, role, email }, JWT_SECRET, { expiresIn: "8h" })`
4. Response: { token, user } (password_hash stripped)
5. Frontend: `localStorage.setItem("token", token)`
`localStorage.setItem("user", JSON.stringify(user))`
6. AuthContext: stores user in React state – triggers role-based redirect
 - `role="employee" != /employee/dashboard`
 - `role="hr" != /hr/dashboard`

Request Authentication

```
// axios.js interceptor – runs before every request:
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token")
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Response interceptor – 401 on any non-login endpoint = force logout:
api.interceptors.response.use(null, (err) => {
  if (err.response?.status === 401 && !isLoginAttempt) {
    localStorage.removeItem("token")
    window.location.href = "/login"
  }
})
```

Password Reset (OTP Flow)

```
Step 1 – POST /api/auth/forgot-password { email }
  !' generates 6-digit OTP
  !' bcrypt.hash(otp, 10) stored in users.reset_token
  !' expiry stored in users.reset_expires (now + 10 min)
  !' sends HTML email via Nodemailer/SMTP

Step 2 – POST /api/auth/verify-otp { email, otp }
  !' bcrypt.compare(otp, reset_token)
  !' validates expiry – returns 200 or error

Step 3 – POST /api/auth/reset-password { email, otp, new_password }
  !' re-validates OTP + expiry
  !' bcrypt.hash(new_password, 10) != UPDATE users
  !' clears reset_token and reset_expires
```

Frontend Route Guards

ProtectedRoute.jsx wraps all authenticated routes and checks both authentication state and role:

```
// Two layers of protection:

// 1. ProtectedRoute – in React Router tree
<Route element={<ProtectedRoute role="employee" />}>
  <Route path="/employee/*" element={...} />
</Route>

// 2. AuthContext – if token is missing or user is null !' /login
// 3. Role check – if user.role !== required role !' /login

// On app load: GET /api/auth/me validates the stored token
// If the token is expired !' 401 !' interceptor clears localStorage
```

8. Frontend Architecture

React 18, routing, state management, Axios

Application Bootstrap

```
// main.jsx
ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </BrowserRouter>
)

// App.jsx – global components mounted OUTSIDE routes:
<AuthProvider>
  <VerificationModal />    !• always active, any route
  <Toaster />              !• react-hot-toast
  <Routes>
    <Route path="/login" element={<Login />} />
    <Route element={<ProtectedRoute role="employee" />}>
      <Route element={<EmployeeLayout />}>
        <Route path="/employee/dashboard" element={<EmpDashboard />} />
        ...
      </Route>
    </Route>
    <Route element={<ProtectedRoute role="hr" />}>
      ...
    </Route>
  </Routes>
</AuthProvider>
```

State Management

KiwiTrack uses React's built-in Context API — no Redux or Zustand. Two contexts:

Context	State	Key Methods
AuthContext	user, loading, token	login(email,pw), logout(), automatically fetches /api/auth/me on mount
ThemeContext	isDark (bool)	toggleTheme() — writes to localStorage, sets class on <html>

Vite Dev Proxy

The Vite dev server proxies all /api requests to the Express backend, avoiding CORS issues during development:

```
// vite.config.js
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://localhost:5000",
        changeOrigin: true,
      }
    }
  }
})
```

9. Core Features

Attendance, Leave, Permission, Payslip, Holidays

Attendance System

The attendance system supports multiple check-in/check-out pairs per day. Each pair is stored in `attendance_logs`; the parent attendance row caches the aggregated totals.

- Check-in: validates no open check-in exists, inserts log, recomputes status from first check-in time, triggers verification scheduling
- Check-out: validates open check-in exists, inserts log, recomputes `total_working_minutes` from all log pairs, re-evaluates status
- Status engine: present !' on-time; late !' after `late_threshold`; half_day !' after `half_day_threshold` OR `<min_full_day_minutes` worked
- Special statuses (`on_leave`, `holiday`, `weekend`) are never overwritten by the status engine
- Monthly view: groups all logs by day, shows first check-in and last check-out with full drill-down detail

Leave Management

- Leave types: Casual Leave, Sick Leave, Earned Leave etc. — configurable via HR (`max_per_year`, `is_paid`)
- Balances: allocated per employee per year — deducted atomically on HR approval
- Overlap check: prevents two pending/approved requests covering the same date range
- Approval cascade: sets attendance status to "on_leave" for every working day in the approved range
- Working day calculation: skips Saturday/Sunday in the days count when applying

Permission Requests

- Short-duration absence (e.g. 2 hours for a doctor visit)
- HR configures `max_permissions_per_month` and `max_permission_hours` limits
- Approval/rejection workflow identical to leave — audit logged

Payslip Management

- HR uploads PDF payslips via multipart form — stored in `uploads/payslips/{userId}/`
- Multer handles file validation: PDF only, max 10MB
- Employees see their own list and can stream-download any payslip
- Metadata (`month`, `year`, `net_pay`, `file_path`) stored in `payslips` table
- Unique constraint (`user_id`, `month`, `year`) prevents duplicate uploads — HR can re-upload to replace

Holiday Calendar

- Stored in `holidays` table — `date`, `name`, `type` (`national/gazetted/restricted`), `category`
- HR can add holidays manually or use the "Sync Indian Holidays" feature
- `indianHolidays.js` contains a static list of Indian government holidays — injected for any given year
- Attendance rows for holiday dates are automatically marked `status="holiday"` during daily processing

10. Verification Checkout

System

Random popup, OS notifications, Service Worker

Overview

The Verification Checkout System is KiwiTrack's core attendance integrity feature. During each work session, the system randomly schedules 2-3 silent check-out events. The employee must respond within the grace period by clicking "Check In" — proving they are actively at their workstation.

Session Windows

Setting	Default	Description
Session 1 Start/End	10:00 – 13:00	Morning session — configurable in HR Settings
Session 2 Start/End	14:30 – 18:30	Afternoon session — configurable in HR Settings
Exclude Start Buffer	15 min	No triggers in first 15 min of each session
Exclude End Buffer	15 min	No triggers in last 15 min of each session
Checks per Session	2-3	Configurable — triggers generated per session
Min Gap	30 min	Minimum minutes between two triggers
Grace Period	5 min	Time employee has to respond before "no response"

Trigger Generation Algorithm

verificationHelper.js uses a seeded pseudo-random number generator (Mulberry32) to generate trigger times. The seed is derived from `userId + date + sessionNumber` — making the schedule deterministic and reproducible but not guessable by the employee.

```
// Deterministic seed per employee/day/session:
seedStr = `${userId}-${dateStr}-${sessionNum}`
seedNum = djb2_hash(seedStr)
rand     = mulberry32(seedNum)

// Usable window = [sessionStart + excludeStart, sessionEnd - excludeEnd]
// Pick N random minutes within that range with minGap constraint
// Insert N rows into verification_checks with status="pending"
```

Poll !' Popup !' Check In Flow

```

Every 30s: GET /api/verification/poll
  % % % Check verification_enabled = "1"
  % % % Expire overdue pending checks (grace period elapsed)
  % % % Find earliest pending check where scheduled_time <= NOW
  %   AND actual_checkout_time IS NULL
  % % % Verify employee is currently checked in (last log = check_in)
  % % % Insert auto check_out log at scheduled_time
  % % % Recompute attendance totals
  % % % Set actual_checkout_time on verification_checks row
  % % % Return { triggered: true, verificationId, scheduledTime, session, gracePeriodMinutes }

```

Frontend receives triggered=true:

```

! ' showPopup() sets React state (phase="pending")
! ' playNotificationBeep() (notification.wav at volume 1.0)
! ' sendOsNotification() via Service Worker (always fires)
! ' SW shows OS toast with "" Check In" action button

```

Employee clicks "Check In" (in-app OR OS notification button):

```

! ' POST /api/verification/respond { verificationId }
! ' Records checkin_time, response_delay_seconds
! ' Inserts check_in log (resuming attendance)
! ' status = "responded"

```

Service Worker Architecture

public/sw.js handles OS notification clicks. This is required because `window.focus()` is blocked in regular page scripts — only the SW's `notificationclick` event is a trusted context where `clients.focus()` always works.

// sw.js – three event listeners:

```

// 1. install ! ' skipWaiting() (immediate activation)
// 2. activate ! ' clients.claim() (controls all tabs instantly)

```

// 3. notificationclick:

```

event.action === "checkin"

```

```

! ' fetch("/api/verification/respond", { JWT from Cache API })
! ' appClient.postMessage({ type: "VERIFICATION_CHECKED_IN_FROM_NOTIFICATION" })

```

```

event.action === "" (body click)

```

```

! ' clients.matchAll() ! ' find our tab
! ' appClient.focus() ! • works cross-app (trusted context)
! ' appClient.postMessage({ type: "SHOW_VERIFICATION_POPUP", ...data })

```

// JWT is cached in Cache API by the page:

```

// caches.open("kiwitrack-auth") ! ' cache.put("/__sw_token__", token)
// SW retrieves it during notificationclick to authenticate the API call

```

12. Data Flow & Request Pipeline

End-to-end request lifecycle

A Typical API Request Lifecycle

1. User action (e.g. clicks Check In button)
%% React component calls: `api.post("/attendance/check-in")`
2. Axios interceptor (`axios.js`)
%% Adds Authorization: Bearer <JWT> header
3. Vite dev proxy (`vite.config.js`)
%% `/api/*` !' `http://localhost:5000/api/*`
4. Express CORS middleware
%% Validates Origin: `http://localhost:5173`
5. Express route matching
%% `POST /api/attendance/check-in` !' `attendance.js` router
6. auth middleware (`auth.js`)
%% Verifies JWT !' loads user from DB !' attaches `req.user`
7. Route handler
%% Business logic !' `db.getAsync` / `db.runAsync` queries
8. Response
%% `res.json({ message, data })`
9. Axios response interceptor
%% On 401: clears `localStorage` !' redirects to `/login`
10. React component
%% Updates state !' re-renders UI

Check-In Data Flow Detail

```
POST /api/attendance/check-in
! "
Check last attendance_log for today: must be check_out (or none)
! "
ensureAttendanceRow(userId, date) !• INSERT OR reuse
! "
INSERT attendance_logs (action="check_in", timestamp=now)
! "
SELECT all logs for today !' computeWorkingMinutes()
! "
computeStatus(firstCheckIn, settings) !' "present" | "late" | "half_day"
! "
UPDATE attendance SET first_check_in, check_in, total_working_minutes, status
! "
scheduleVerification(userId, date, timeHHMM) !• async, fire-and-forget
! "
Read session windows from company_settings
Find matching window for current time
Generate N random trigger times (Mulberry32 seeded RNG)
INSERT verification_checks rows with status="pending"
```

13. Environment & Deployment

Setup, environment variables, npm scripts

Environment Variables (.env)

Variable	Required	Description
PORT	No (5000)	Express server port
JWT_SECRET	YES	Secret key for JWT signing — use a strong random string in production
JWT_EXPIRES_IN	No (8h)	Token lifetime — e.g. "8h", "1d", "7d"
SMTP_HOST	For email	SMTP server hostname — e.g. smtp.gmail.com
SMTP_PORT	For email	SMTP port — 587 for STARTTLS, 465 for SSL
SMTP_USER	For email	SMTP username / email address
SMTP_PASS	For email	SMTP password or Gmail App Password
UPLOAD_DIR	No	Base directory for file uploads — defaults to "uploads"

Setup from Scratch

```
# 1. Clone repo
git clone <repo> && cd attendance-system

# 2. Backend setup
cd backend
npm install
cp .env.example .env           # fill in JWT_SECRET and SMTP values
node db/migrate.js             # creates base tables
node db/migrate_v2.js          # adds attendance_logs, audit_logs
node db/migrate_v3.js          # adds verification_checks
node db/migrate_v4.js          # adds login_history, settings expansion
node db/migrate_v5.js          # adds session window settings
node db/seed.js                # (optional) inserts demo employees
npm run dev                    # starts Express on port 5000

# 3. Frontend setup
cd ../frontend
npm install
npm run dev                    # starts Vite on port 5173

# 4. Open http://localhost:5173
```

NPM Scripts

Location	Script	Command	Purpose
backend	npm start	node server.js	Production start — no auto-reload
backend	npm run dev	nodemon server.js	Development — auto-reloads on save
backend	npm run migrate	node db/migrate.js	Run base schema migration
backend	npm run seed	node db/seed.js	Insert development seed data
frontend	npm run dev	vite --port 5173	Dev server with HMR
frontend	npm run build	vite build	Production build to /dist
frontend	npm run preview	vite preview	Preview production build locally

Production Considerations

- Serve the Vite build (/dist) via Nginx or Express static — remove the Vite proxy
- Set CORS origin in server.js to the actual production domain
- Use a proper secrets manager for JWT_SECRET in production (not .env file)
- For notifications to work outside localhost, the site must be served over HTTPS — Service Workers require a secure context
- SQLite is suitable for single-server deployments. For multi-server scaling, migrate to PostgreSQL
- Back up attendance.db regularly — it is the only persistent data store
- Nodemailer requires a valid SMTP provider for password reset OTP delivery (Gmail App Passwords work well)

KiwiTrack

End of Documentation

This document covers the complete technical specification of the KiwiTrack Attendance Management System as of September 2026.

For questions, refer to the inline code comments or the relevant route handler in `/backend/routes/`.

